API Security Gateway

with Automated Rate Limiting and Token Validation

Project Specification, Research & Implementation Plan — Major Project, B.Tech CSE (AI)

Prepared by	Panchireddy Vasavi
Program	B.Tech CSE (AI), Expected Graduation 2027
Institution	International School of Technology and Sciences, Rajanagaram (JNTU)
Domain Focus	Cybersecurity / Application Security

1. Abstract

Modern applications expose their functionality through APIs, making the API layer the primary attack surface for credential stuffing, token replay, brute-force, and denial-of-service style abuse. This project designs and implements an **API Security Gateway** that sits in front of backend services and enforces two complementary controls before any request reaches business logic: **automated rate limiting** (to throttle abusive or excessive traffic per client) and **token validation** (to verify the authenticity, integrity, and scope of JSON Web Tokens issued through an OAuth 2.0 / OpenID Connect flow). The gateway is built as a standalone reverse-proxy style middleware so that it can be dropped in front of any REST API without modifying downstream services. Rate limiting is adaptive rather than fixed: limits are configurable per client, per endpoint, and per subscription tier, and are enforced using the sliding-window and token-bucket algorithms with Redis as a shared, low-latency counter store. Token validation covers signature verification, expiry, audience/issuer checks, revocation-list lookups, and scope-based authorization. The system is designed, implemented, tested against common attack scenarios (replay attacks, brute-force login attempts, burst traffic), and evaluated for latency overhead, throughput, and false-positive/false-negative rates on legitimate versus malicious traffic.

2. Problem Statement

APIs are increasingly the backbone of web, mobile, and IoT applications, and are also the fastest-growing attack vector according to industry threat reports. Two of the most common and damaging classes of API abuse are:

- **Uncontrolled request volume:** attackers or misbehaving clients flood an API with requests, exhausting compute resources, degrading service for legitimate users, or brute-forcing credentials and OTPs through sheer repetition.
- **Weak or absent token verification:** services that trust a bearer token without fully validating its signature, expiry, audience, and scope are vulnerable to token replay, privilege escalation, and forged-token attacks.

Individually, many APIs implement ad-hoc, inconsistent versions of these controls duplicated across services, which increases maintenance cost and the chance of a misconfigured or forgotten check. This project addresses the problem by centralizing both controls into a single, reusable gateway component that every request must pass through, ensuring consistent enforcement, centralized logging/auditing, and a single point of policy configuration.

3. Objectives

- Design a gateway architecture that intercepts all inbound API traffic before it reaches backend services.
- Implement automated, configurable rate limiting using industry-standard algorithms (Token Bucket and Sliding Window Log/Counter) with per-client and per-endpoint granularity.
- Implement robust JWT-based token validation, including signature verification (RS256/HS256), expiry, issuer/audience checks, and revocation handling.
- Provide adaptive throttling that reacts to abnormal traffic patterns (e.g., temporarily tightening limits for a client showing brute-force-like behavior).
- Build centralized logging, monitoring, and alerting for blocked requests, invalid tokens, and rate-limit violations.

- Benchmark the gateway's added latency and throughput impact and demonstrate resilience against simulated attack traffic.

4. Scope

Category	Description
In Scope	Gateway service, rate limiting engine, JWT/OAuth2 token validation, policy configuration, logging & monitoring dashboard, load/attack simulation and testing.
Out of Scope	Building the backend business-logic APIs themselves (mock/demo APIs are used as protected targets), full production-grade identity provider (a lightweight auth/token-issuing service is simulated), mobile client development.

5. Research and Literature Review

5.1 API Security Landscape

The OWASP API Security Top 10 identifies broken authentication, unrestricted resource consumption, and broken object/function-level authorization as leading API risk categories. Unrestricted resource consumption directly corresponds to the absence or misconfiguration of rate limiting, while broken authentication corresponds to weak token issuance or validation. This project is scoped to directly address both categories, using OWASP's guidance as the baseline threat model.

5.2 Rate Limiting Algorithms

Algorithm	Mechanism	Advantage	Limitation
Fixed Window Counter	Counts requests in fixed time slots (e.g., per minute).	Simple, low memory.	Bursts at window boundaries can double the effective limit.
Sliding Window Log	Stores timestamps of each request and counts those within a rolling window.	Very accurate, no boundary burst issue.	Higher memory use for high-traffic clients.
Sliding Window Counter	Approximates the sliding log using weighted counts from adjacent fixed windows.	Good accuracy, low memory.	Slight approximation error.
Token Bucket	Tokens refill at a fixed rate; a request consumes a token; requests are rejected when the bucket is empty.	Naturally supports bursts up to bucket size; smooth long-term rate.	Requires careful tuning of bucket size and refill rate.
Leaky Bucket	Requests are queued and processed at a constant outflow rate.	Smooths traffic to a constant rate.	Added latency/queueing; less suited to bursty legitimate traffic.

Based on this comparison, the project adopts the **Token Bucket** algorithm as the primary limiter (for its natural handling of legitimate bursts) combined with a **Sliding Window Counter** as a secondary, longer-horizon guard against sustained abuse — a hybrid approach used by several production API gateways.

5.3 Token-Based Authentication and Validation

JSON Web Tokens (JWT), standardized in RFC 7519, are the dominant format for stateless API authentication. A JWT consists of a header, payload (claims), and a cryptographic signature. Correct validation requires more than confirming the signature: the gateway must also check the **exp** (expiry), **nbf** (not-before), **iss** (issuer), **aud** (audience), and any custom scope/role claims, and must consult a revocation mechanism (e.g., a denylist or short-lived token strategy) since JWTs are not inherently revocable once issued. The project follows the OAuth 2.0 (RFC 6749) framework for token issuance semantics and OpenID Connect conventions for claim structure, and treats the gateway as a "resource server" that never trusts a token without full local verification.

5.4 Related Tools and Prior Art

Production systems such as Kong, NGINX with rate-limiting modules, AWS API Gateway, and Envoy Proxy implement similar controls at scale. Studying their configuration models (e.g., Kong's plugin-based rate-limiting and JWT plugins) informs the design of this project's policy-configuration layer, while the actual implementation is built independently and at a scope

appropriate for an academic major project.

6. Threat Model

Threat	Description	Mitigation via Gateway
Credential brute-force / stuffing	Repeated login attempts against an auth endpoint.	Per-IP and per-account rate limiting with exponential backoff after repeated failures.
Token replay	A captured valid token is reused after it should no longer be trusted.	Short token expiry, revocation-list checks, optional single-use nonce for sensitive operations.
Forged/tampered token	An attacker crafts or modifies a token payload.	Mandatory signature verification (RS256) rejecting any token with invalid signature.
Volumetric / burst flooding (L7 DoS)	A client or botnet sends excessive requests to exhaust resources.	Token Bucket + Sliding Window limiting, with adaptive tightening on anomaly detection.
Scope / privilege escalation	A valid but low-privilege token used to access a restricted endpoint.	Scope and role claim verification against each endpoint's required permissions.

7. System Architecture

The gateway is implemented as a reverse-proxy middleware layer positioned between clients and backend APIs. Every inbound request passes through an ordered pipeline of checks before being forwarded upstream.

7.1 Request Pipeline

- **1. TLS Termination & Request Parsing** — the gateway receives the HTTPS request and extracts client identity signals (IP, API key, or bearer token).
- **2. Rate Limit Check** — the client/endpoint identifier is looked up in the Redis-backed counter store; the request is throttled (HTTP 429) if limits are exceeded, before any authentication work is done, to cheaply reject abusive traffic early.
- **3. Token Extraction & Validation** — the Authorization header is parsed, the JWT signature is verified against the identity provider's public key (JWKS), and expiry/issuer/audience/scope claims are checked; revoked tokens are rejected via a Redis denylist lookup.
- **4. Authorization Decision** — validated claims are matched against the target endpoint's required scopes/roles.
- **5. Request Forwarding** — on success, the request is proxied to the appropriate backend service with verified identity context attached in a trusted header.
- **6. Logging & Metrics** — every decision (allowed, rate-limited, unauthorized) is logged with timestamp, client identity, and reason, feeding a monitoring dashboard.

7.2 Component Overview

Component	Responsibility
API Gateway Core	Request interception, pipeline orchestration, response shaping (429 / 401 / 403).
Rate Limiter Module	Implements Token Bucket + Sliding Window Counter; reads/writes counters in Redis.
Token Validator Module	JWT parsing, signature verification via JWKS, claims validation, revocation check.
Policy Store	Configuration of per-client tiers, per-endpoint limits, and required scopes (JSON/YAML or a small database table).
Identity Provider (simulated)	Issues JWTs after mock login, exposes a JWKS endpoint for public-key verification, supports token revocation.

Component	Responsibility
Redis Cache	Shared, low-latency store for rate-limit counters and the token revocation/denylist.
Logging & Monitoring	Structured logs plus a metrics dashboard (e.g., Grafana/simple web dashboard) showing traffic, block rates, and latency.
Demo Backend APIs	Mock protected services (e.g., a "users" and "orders" API) used to demonstrate end-to-end enforcement.

7.3 Data Flow Summary

Client → Gateway (rate-limit check → token validation → authorization) → Backend API → Response returned through the Gateway → Response and decision metadata logged. Rejections at the rate-limit or token-validation stage short-circuit the pipeline and never reach the backend, minimizing load on protected services during an attack.

8. Proposed Technology Stack

Layer	Technology	Rationale
Gateway runtime	Node.js (Express) or Python (FastAPI)	Fast to develop, strong middleware support, good JWT/OAuth libraries.
Rate-limit / revocation store	Redis	In-memory, sub-millisecond operations, native TTL support ideal for counters and denylists.
Token format & standards	JWT (RFC 7519), OAuth 2.0 (RFC 6749), OpenID Connect claims	Industry standard, broad library support.
Token libraries	jsonwebtoken / python-jose / PyJWT	Well-tested signature verification and claims handling.
Reverse proxy layer (optional prod-style)	NGINX in front of the gateway	TLS termination and coarse-grained protection ahead of app-level logic.
Containerization	Docker / Docker Compose	Reproducible local deployment of gateway + Redis + demo APIs.
Load / attack simulation	Locust or k6, custom Python scripts	Simulate burst traffic, brute-force login attempts, and token replay for testing.
Monitoring	Grafana + Prometheus, or a lightweight custom dashboard	Visualize request rates, block rates, and latency.
Version control & docs	Git/GitHub	Source control, issue tracking, project documentation.

9. Implementation Plan

The project is broken into six phases over a typical major-project timeline (adjustable to the institution's semester schedule). Each phase produces a concrete, demonstrable deliverable.

Phase / Weeks	Key Activities	Deliverable
Phase 1 Requirements & Design (Weeks 1–2)	Finalize threat model, define rate-limit policies and token claim schema, design pipeline architecture and data flow diagrams.	Architecture document, sequence diagrams, policy schema.
Phase 2 Core Gateway & Proxying (Weeks 3–4)	Build the base reverse-proxy service that forwards requests to demo backend APIs; set up Docker Compose environment with gateway, Redis, and mock backend services.	Working pass-through gateway with logging.

Phase / Weeks	Key Activities	Deliverable
Phase 3 Rate Limiting Engine (Weeks 5–6)	Implement Token Bucket and Sliding Window Counter algorithms against Redis; add per-client and per-endpoint policy configuration; return proper 429 responses with Retry-After headers.	Rate limiter module with unit tests and configurable policies.
Phase 4 Token Validation & Authorization (Weeks 7–8)	Stand up a lightweight identity provider (mock login + JWKS endpoint); implement signature verification, expiry/issuer/audience checks, scope-based authorization, and a Redis-backed revocation list.	Token validator module, working login-to-protected-resource flow.
Phase 5 Adaptive Throttling, Logging & Monitoring (Weeks 9–10)	Add anomaly-based adaptive tightening (e.g., temporary stricter limits after repeated auth failures), structured logging of all decisions, and a metrics dashboard.	Monitoring dashboard, adaptive rules, centralized logs.
Phase 6 Testing, Attack Simulation & Report (Weeks 11–13)	Run load tests, brute-force simulations, and token-replay tests; measure latency overhead and throughput; fix issues; finalize documentation and prepare demo.	Test report with metrics, final documentation, live demo.

9.1 Milestone Timeline (13-Week View)

Week(s)	Milestone
1–2	Design & architecture finalized
3–4	Base gateway proxying live traffic
5–6	Rate limiting enforced end-to-end
7–8	Token validation & authorization enforced
9–10	Adaptive throttling + monitoring dashboard live
11–12	Attack simulations & performance benchmarking complete
13	Final report, documentation, and demo ready

10. Testing and Evaluation Plan

10.1 Functional Testing

- Valid token + within rate limit → request forwarded successfully (200 OK).
- Valid token + rate limit exceeded → HTTP 429 with Retry-After header, request not forwarded.
- Expired, malformed, or badly signed token → HTTP 401, request not forwarded.
- Valid token but insufficient scope for the endpoint → HTTP 403.
- Revoked token presented again → HTTP 401 via denylist check.

10.2 Security / Attack-Simulation Testing

Scenario	Method	Expected Result
Brute-force login simulation	Scripted repeated login attempts against the auth endpoint.	Requests throttled after configured threshold; account/IP temporarily locked out.
Token replay attempt	Reuse a previously revoked or expired token.	Request rejected with 401.
Burst traffic / flood test	Sudden spike of requests from a single client using Locust/k6.	Excess requests rejected with 429; backend load remains stable.

Scenario	Method	Expected Result
Distributed low-rate abuse	Many distinct IPs each sending traffic just under the per-IP limit.	Documented as a known limitation; discuss mitigation via device fingerprinting / account-level limits.

10.3 Performance Evaluation

The gateway's added latency (time spent in rate-limit check + token validation, excluding backend processing) and maximum sustainable throughput (requests/second before error rates rise) are measured with and without the gateway in place, to quantify the security-vs-performance trade-off. Target benchmarks: added latency under 15–20 ms per request at moderate load, and graceful, predictable degradation (via 429 responses rather than crashes) under overload.

11. Expected Outcomes

- A working, demonstrable API Security Gateway enforcing both rate limiting and token validation on sample protected APIs.
- Quantified evidence that the gateway blocks brute-force, replay, and flood-style attacks while adding minimal latency to legitimate traffic.
- A reusable, configuration-driven policy model that could plausibly be extended to real microservice deployments.
- A complete project report, architecture diagrams, and monitoring dashboard suitable for major-project evaluation and for a cybersecurity-focused portfolio/resume.

12. Limitations and Future Scope

- Per-IP/per-client limiting can be bypassed by distributed low-and-slow attacks; future work could add behavioral/ML-based anomaly detection.
- The identity provider is simulated for this project; production use would integrate a full OAuth2/OIDC provider (e.g., Keycloak, Auth0).
- Current design assumes a single-region Redis instance; a production system would need a distributed, highly available rate-limit store.
- Future extensions: mutual TLS between gateway and backends, GraphQL-aware rate limiting, and an ML-based anomaly scoring layer for adaptive throttling.

13. References

- OWASP Foundation, "OWASP API Security Top 10."
- IETF RFC 7519 — "JSON Web Token (JWT)."
- IETF RFC 6749 — "The OAuth 2.0 Authorization Framework."
- IETF RFC 6750 — "The OAuth 2.0 Authorization Framework: Bearer Token Usage."
- Kong Inc., NGINX Inc., and Envoy Proxy official documentation on rate-limiting and JWT-validation plugins.
- Redis official documentation on TTL-based counters and rate-limiting patterns.